

Biomedical Engineering

Final Year Project

Templating in Standard ML

Ja Ying Lew

May 2026

Supervisor: Gergely Buday

Dissertation submitted to the University of Sheffield in partial fulfilment
of the requirements for the degree of

(Bachelor of Engineering)

Abstract

The first item in your dissertation should be an abstract of a few short paragraphs (200 words maximum) summarising the dissertation. This should not be a re-statement of the original project outline. A suggested flow is background, project aims and main achievements. From the abstract a reader should be able to ascertain if the project is of interest to them and presents results that they would like to know the details of.

Acknowledgements

You may wish to include an acknowledgement. This should be a maximum of one page and is usually much less. Thank whoever may have helped you in any way , both serious and a bit of fun.

The acknowledgements are written in the first person singular, i.e. “I would like to thank my ...”

Contents

Abstract	i
Acknowledgements	ii
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Relationship to the Degree Programme	2
2 Literature Review	3
2.1 Theoretical Framework	3
2.2 Template System Design	3
2.3 Templating Engines Across Platforms	4
2.4 Alternative Templating Systems	5
2.5 Functional Languages for Template Processing	6
2.6 Takeaway	8
3 Requirements and Analysis	10
3.1 Problem Statement	10
3.2 Functional Requirements	10
3.3 Non-Functional Requirements	11
3.4 Problem Decomposition	12
3.4.1 Parsing the Mustache Template	12
3.4.2 Enriching the JSON Data	12
3.4.3 Rendering the Template	13
3.4.4 CLI and Build System	13
3.4.5 Testing	13
3.5 Known Gaps and Limitations	14
3.6 Evaluation Strategy	14
3.7 Legal and Ethical Considerations	15
4 Design	16

4.1	Choice of Implementation Language	16
4.2	Architectural Choice: Preprocessor Rather Than New Engine	17
4.3	Parser Design	18
4.3.1	Parser Generators	18
4.3.2	AST Shape	18
4.4	Preprocessor Design	19
4.4.1	Operating on the JSON Datatype	19
4.4.2	Public Interface	19
4.4.3	A Worked Example	20
4.5	Renderer Design	21
4.5.1	Tree-Walking Interpreter	21
4.5.2	Truthiness	21
4.5.3	Dot-Separated Path Resolution	21
4.6	Build and Deployment	22
5	Implementation and Testing	23
5.1	Implementation Overview	23
5.2	Implementing the Parser	23
5.3	Implementing the Preprocessor	24
5.4	Implementing the Renderer	25
5.5	Coding Traps Encountered	25
5.6	Testing Strategy	26
5.7	Test Results	27
5.8	Limitations Confirmed by Testing	27
6	Results and Discussion	28
6.1	Summary of Findings	28
6.2	Goals Achieved	28
6.3	Comparative Analysis	29
6.4	Discussion	30
6.5	Further Work	31
7	Conclusions	33
A	Appendix Title	35

List of Figures

List of Tables

2.1	Template System Comparison (Author’s analysis applying Parr’s framework [1] and Pisu’s findings [2])	9
-----	--	---

1. Introduction

Templating is one of the most common operations in modern software. Any web application that produces dynamic HTML, any system that generates emails or documents from a database, and any tool that produces source code from a specification, will, at some point, separate a fixed structural skeleton from the variable data that fills it in. This separation is what a template system provides.

For most readers approaching this dissertation from a biomedical engineering background, the closest familiar example is probably scientific report generation: a Jupyter notebook that produces a formatted report from experimental data, or a LaTeX template that pulls in measurements from a CSV file. The same idea, applied at scale, is what powers the dynamic content on most of the websites a modern person interacts with daily.

The catch is that template systems exist on a spectrum. At one end are “logic-friendly” systems that allow arbitrary code to run inside the template, which is convenient but exposes the application to a class of security attacks called server-side template injection. At the other end are “logic-less” systems, of which the most well-known is Mustache, which forbid any logic in the template at all. Mustache is significantly safer by construction, and the same Mustache template can be rendered by implementations in dozens of programming languages, which makes it an appealing choice for cross-language tooling.

The price of this safety is what the literature calls the *preprocessing burden*. Anything the template would have computed itself in a logic-friendly system, the developer must now compute in advance and embed in the data. If a list needs commas between items but not after the last one, the developer has to walk the list manually and attach an `isLast` flag to each element. If a field might be missing, the developer has to add a companion `has_field` flag to make the template’s conditional render correctly. None of this is hard, but all of it is tedious, error-prone, and untyped. Nothing checks that the developer remembered to add every flag, and a missing flag is silent at compile time and only visible by inspecting the rendered output.

This dissertation investigates whether Standard ML, a strictly typed functional language, can automate that preprocessing while preserving Mustache’s logic-less guarantees. Functional languages have a long history of language-processing applications, including in scientific computing. BioCaml, for example, is a widely-used bioinformatics library written in OCaml. The properties that make these languages useful for parsing and transformation work: algebraic datatypes that naturally express tree-shaped data, pattern matching that the compiler checks for completeness, and a strong type system that catches errors before runtime, are

also the properties that make them well-suited to preprocessing automation.

The contribution of this project is a working prototype. It consists of a Mustache parser, a JSON preprocessor that adds two classes of contextual metadata (null-handling flags and array position flags), and a renderer that consumes the enriched data. The preprocessor sits in front of any standard Mustache implementation, requires no changes to the Mustache specification, and uses Standard ML's pattern-matching exhaustiveness check to ensure that every transformation is applied consistently across nested data structures. All 47 unit tests across the three components pass, and the tool runs end-to-end from the command line.

Honest scope: the prototype works on the JSON shapes it was designed for. It has not been stress-tested against unusual or malformed inputs, two parts of the Mustache specification (partials and delimiter changes) are parsed but not rendered, and several deployment quirks make the tool harder to install than necessary. These limitations are documented in detail in Chapter 3 and revisited in the conclusion. The contribution is not a production-ready tool; it is evidence that the typed-functional approach to preprocessing automation is viable, with clear directions for the work needed to make it deployable at scale.

1.1 Relationship to the Degree Programme

This project is a Bachelor of Engineering dissertation in Biomedical Engineering, which may seem distant from a software-tooling problem about template systems. The connection is that the engineering disciplines all rely on scientific programming, and that scientific programming is increasingly carried out in functional languages where correctness guarantees matter. BioCaml, the Coq proof assistant (written in OCaml), and the formally-verified compilers used in safety-critical medical-device firmware are all examples of typed functional languages applied to engineering problems where errors have real consequences.

For this particular application of SML, I enjoy how biomedical engineers are so interdisciplinary and I feel like this would link in the long run?

2. Literature Review

2.1 Theoretical Framework

This review evaluates template systems through Parr’s (2004) formal framework for model-view separation, which defines five "entanglement" violations: conditional logic based on view structure, method/function calls from templates, value computation within templates, model scope manipulation, and recursive rendering [1]. Parr shows that unrestricted templates inevitably violate separation when developers embed business logic, as templates end up producing logic, with template computational power classified from weakest (substitution-only) to strongest (Turing-complete). Pisu et al.’s (2024) security analysis confirms these theoretical predictions: examining 34 template engines, they found 71% of server-side template injection (SSTI) vulnerabilities occur in engines permitting conditional logic, with Jinja2’s sandbox suffering documented bypasses [2]. This highlights our core limitation: strict separation (Mustache) enforces security but imposes preprocessing burden; unrestricted power (Jinja2, EJS) reduces preprocessing but enables separation violations and security exploits.

2.2 Template System Design

Template systems separate presentation markup from application code through ‘separation of concerns’: business ‘model’ logic (data processing, calculations) remains distinct from presentation ‘view’ logic (display structure). This separation improves maintainability, reduces security vulnerabilities, and enables parallel development by allowing designers to modify presentation independently from programmers updating business logic [1].

Effective template systems must satisfy three key design criteria. First, they should enforce separation of concerns, ensuring business logic remains in the model rather than templates. Second, they must provide clear syntactic boundaries that distinguish presentation structure from application data, especially for their target language. Third, they must balance computational power appropriately.

This balance is critical. Parr (2004) demonstrates that template engines with greater computational power (those allowing conditionals, arithmetic operations, and method calls) enable logic duplication and violate model-view separation when developers embed business rules directly in templates under deadline pressure [1]. The security implications are severe, particularly the analysis of 34 templates of which 29% provide security features, with most vulnerabilities stemming from excessive computational power in templates [2]. However, overly restrictive systems present their own challenges, requiring substantial data preprocessing before rendering and producing view models with excessive getter methods [3], [4].

The ideal template system restricts computational power to only what is necessary for presentation while avoiding unnecessary complexity in the model layer.

2.3 Templating Engines Across Platforms

The architectural tension between logic-friendly and logical design recurs across every major programming platform. Specifically Parr identifies Entanglement Index ranging from 1 to 5 of which template engines are usually polarised to either Index 1 (strictly separated) or Index 5 (Totally entangled). Thus, it can be identified that the problem is the tension of this balance [1].

In the PHP (Hypertext Preprocessor) ecosystem, this problem is particularly visible. Blade, Laravel's native templating language, works to allow developers embed arbitrary PHP expressions in view making the logic entangled. On the other hand, twig was designed with an explicit sandbox that prohibits direct PHP execution, utilising the separation of concerns [5]. These example template engines display how developers consistently trade security and separation against the convenience of unrestricted logic. Ruby also has a similar issue. Embedded Ruby (ERB) allows unrestricted Ruby execution within templates, offering maximum flexibility at the cost of separation. Liquid, developed by Shopify, took a different angle, which they designed specifically to allow untrusted third-party authors to customise e-commerce themes without the compromise of the underlying server. Liquid ensures that template execution is strictly bounded [6]. A significant note is that Shopify came to this logic-less design from a pure security motivation, suggesting independent evidence that restricting template computational power is an architectural choice. The dangers of unstructured templates are not theoretical: Parr (2004) documents that 'SquareSpace' was forced to abandon an unrestricted engine after attackers exploited templates to invoke malicious class loaders [1].

Mustache's widespread adoption of a dozen languages demonstrates a universal demand for a logic-less baseline, independent of any particular language's native templating facilities. However, this logic-less design consistently introduces what developers like to call a 'preprocessing burden', in other words 'data massaging'. Due to the lack of conditionals and complex iteration of Mustache or Liquid, there must be an over prepared data to match the template's exact structure. In the Scala/Play Framework, teams adopting Mustache found themselves creating an entirely new architectural layer, the Presenter, solely to hold logic the template could no longer express [7]. This produces a subtle but significant problem: presentation logic does not disappear; it leaks into the model. A template unable to compute alternating row colours forces the model to carry `isEven` flags or explicit CSS class names, polluting business logic with view concerns and partially undermining the separation the logic-less design was intended to enforce **parr2004**.

This pattern is consistent across platforms and remains unsolved. No existing logic-less sys-

tem provides automated support from the preprocessing transformations it requires. This project addresses that gap directly: by automating common preprocessing patterns in Standard ML, it preserves the security and separation guarantees of logic-less design while eliminating the manual data preparation burden that currently makes it costly to adopt.

2.4 Alternative Templating Systems

Modern HTML Templating Engines:

- **Handlebars** extends Mustache by adding helpers, block expressions, and partials while maintaining logic-less philosophy, compiling templates into JavaScript functions for faster execution [8], [9].
- **Nunjucks**, inspired by Python’s Jinja2, provides block inheritance, autoescaping, macros, and asynchronous control, making it significantly more feature-rich than Mustache while maintaining readable syntax [10]. It offers template inheritance with block overriding and automatic HTML escaping for XSS prevention.

Logic-Friendly Templating Languages:

- **EJS (Embedded JavaScript)** allows developers to write plain JavaScript directly in templates using special delimiters, with `<% %>` for code execution and `<%= %>` for output, eliminating the need to learn new syntax [11]. It caches intermediate functions for fast execution and provides full JavaScript expressiveness with server-side rendering support. However, EJS provides no security boundaries. Templates have direct access to the JavaScript runtime, making it unsuitable for scenarios involving untrusted template sources.
- **Jinja2** is Python’s fast and extensible templating engine that supports template inheritance, macros, HTML autoescaping, and sandboxed environments for safely rendering untrusted templates [4], [12]. It offers more flexibility than Django’s templating system and features a powerful extension system with custom filters and tests. However, the sandbox has documented bypass vulnerabilities and security issues: attackers can exploit Python’s introspection mechanisms (`__class__`, `__subclasses__`) to escape restrictions, with recent exploits (CVE-2024-22195, CVE-2024-27516) demonstrating remote code execution despite sandbox protections [2].

Language-Integrated Templating:

- **React/JSX** integrates markup directly into JavaScript as code through components, where JSX expressions compile to regular JavaScript function calls that can be used in conditionals, loops, and variable assignments [13]. It provides a component-based architecture with full JavaScript expressiveness and seamless TypeScript integration

for type safety.

- **Svelte** shifts template processing from browser runtime to compile-time, generating highly optimized vanilla JavaScript without framework overhead, producing smaller bundles and better performance than runtime frameworks [14]. It eliminates runtime overhead through compile-time processing and provides a reactive programming model with minimal boilerplate code.

These alternatives reveal an inverse relationship: strict separation enforcement correlates with high preprocessing burden (Mustache, Handlebars), while unrestricted computational power correlates with low preprocessing but documented security vulnerabilities (Jinja2, EJS).

2.5 Functional Languages for Template Processing

Functional programming languages (Standard ML, OCaml, and Haskell) bring distinct characteristics to template processing: strong static typing catching errors at compile-time, pattern matching for data structure manipulation, and algebraic datatypes for representing abstract syntax trees [15], [16].

Haskell's lazy evaluation and advanced type system have produced two distinctive templating approaches. Hamlet, part of the Yesod web framework, uses Template Haskell metaprogramming to compile templates at build time, enabling type-checked variable substitution and ensuring that templates referencing undefined fields fail to compile rather than at runtime [17]. This shifts an entire category of template errors from production to compilation, demonstrating how sufficiently powerful type system can guarantee template correctness.

Blaze-html takes a different position, representing HTML directly as composable Haskell functions rather than as separate template language [18]. Templates are not strings to be parsed but rather value constructed through variables. This eliminates the parser entirely and provides maximum type safety, but the cost of abandoning the template-as-text model that makes templates accessible to non-programmers: a tradeoff that mirrors React/JSX's design choices in the JavaScript ecosystem.

OCaml prioritises industrial pragmatism with aggressive compiler optimisations and richer standard library via OPAM [16]; Jingoo provides Jinja2-compatible templating for OCaml applications, allowing developers to use Jinja2 syntax within OCaml programs [19]. While this offers developers familiarity from migrating from python, it inherits Jinja2's security model, including the documented sandbox bypass concerns discussed in the earlier sections, demonstrating that functional languages hosting alone does not resolve the underlying security trade-offs of permissive template languages.

Elm is particularly relevant to this project because it involves strictly functional language

with an existing implementation of the Mustache specification [20]. Elm's design priorities of compile time correctness, explicit data flow and absence of runtime errors, align closely with Mustache's logic-less philosophy: both insist that uncertainty about data shape be resolved before rendering rather than during it. This aligns with what we are aiming for.

The ELM mustache implementation demonstrates an important pattern as Elm is purely functional with no runtime exceptions, the act of supplying data to mustache template becomes an explicit, type-checked transformation. Developers cannot pass partially formed data to a template and discover the gap at render time. The type system requires the data structure to be fully constructed before the template can consume it. This is the preprocessing burden made visible at the type level. Where in JavaScript a missing field produces an empty rendering or silent failure, in Elm it produces a compile error.

The existence of elm-mustache suggests that the pairing of logic-less templates with strictly typed functional languages is a viable architectural choice explored in practice. Elm is able to provide something called the Model-View-Controller: this allows for the viewer to suggest changes via signals/messages to the model, eliminating frontend issues. However, Elm provides no automation of preprocessing transformations, meaning that the developer still has to write the data construction by hand. The type system catches errors of omission but does not eliminate the underlying labour. This identifies the precise space that the project occupies not whether functional languages can host logic less templates but whether preprocessing programmes themselves can be automated within such system, eliminating the manual burden while preserving the type level guarantees.

This project selects Standard ML for several technical reasons. First, Standard ML possesses a complete formal specification with mathematically defined typing rules and operational semantics [21], [22], [23], providing rigor that aligns with this project's focus on compile-time verification. Second, multiple standard-conforming implementations exist (SML/NJ, MLton, MLKit, Poly/ML) [24], ensuring reproducibility across Unix and Windows platforms [25], [26]. Third, Standard ML has proven effectiveness in language processing tasks [27]. Its algebraic datatypes naturally represent template structures and JSON data, while pattern matching enables concise expression of preprocessing transformations: adding boolean flags, inserting separators, and handling format variations. Fourth, the compiler verifies pattern exhaustiveness, ensuring all datatype constructors are handled [21]. For preprocessing automation, this provides compile-time guarantees that all required transformations are applied. As Appel notes, "adding a new constructor to a datatype...will cause compile-time errors everywhere that the pattern matching is incomplete" [27], thus catching omissions before runtime. Finally, Standard ML's design highlights predictability through strict evaluation and a relatively small, coherent language specification [21]. Unlike Haskell's lazy evaluation or OCaml's object system extensions, Standard ML maintains conceptual simplicity while providing the functional programming features essential for this task. However, this

simplicity comes with practical tradeoffs: the ecosystem has significantly fewer third-party libraries than mainstream alternatives [15], requiring custom implementations of facilities like JSON parsing. This project accepts this implementation burden to obtain compile-time verification guarantees unavailable in dynamically-typed alternatives.

The selection of Standard ML for preprocessing automation (rather than template engine implementation) leverages these characteristics: pattern matching naturally expresses transformations like “add `last : true` to final list element,” exhaustive checking ensures all preprocessing cases are handled, and the type system provides compile-time guarantees that required fields are complete, catching omissions before runtime rather than through testing.

2.6 Takeaway

These alternatives reveal distinct positions on the security-usability spectrum. No existing system successfully combines strict separation enforcement with low preprocessing burden: Mustache prioritises security at the cost of extensive manual data preparation [3], [4], while systems like Jinja2 reduce preprocessing but suffer documented security vulnerabilities [2]. While preprocessing requirements are well-documented as complaints, no work formalizes these patterns or provides type-safe automation. Addressing this gap, this project investigates whether Standard ML’s type system can automate preprocessing transformations while preserving logic-less templates’ security advantages.

System	Computational Power	Preprocessing Burden	Security Model	Key Limitation
Mustache	Substitution only	High (all logic pre-computed)	No sandbox needed	Bloated view models
Handlebars	Limited helpers	Medium (helpers reduce burden)	Helper whitelisting	Helper proliferation
Nunjucks	Context-sensitive	Low (templates handle logic)	Sandboxing	Separation violations
Jinja2	Turing-complete	Minimal (full Python access)	Sandbox (bypassable)	SSTI vulnerabilities
EJS	Turing-complete	Minimal (full JavaScript)	None	Direct code injection
React/JSX	Turing-complete	Minimal (native JS)	Type checking only	Not logic-less
Svelte	Compile-time	Low (compiler handles logic)	Static analysis	Requires build step

Table 2.1: Template System Comparison (Author’s analysis applying Parr’s framework [1] and Pisu’s findings [2])

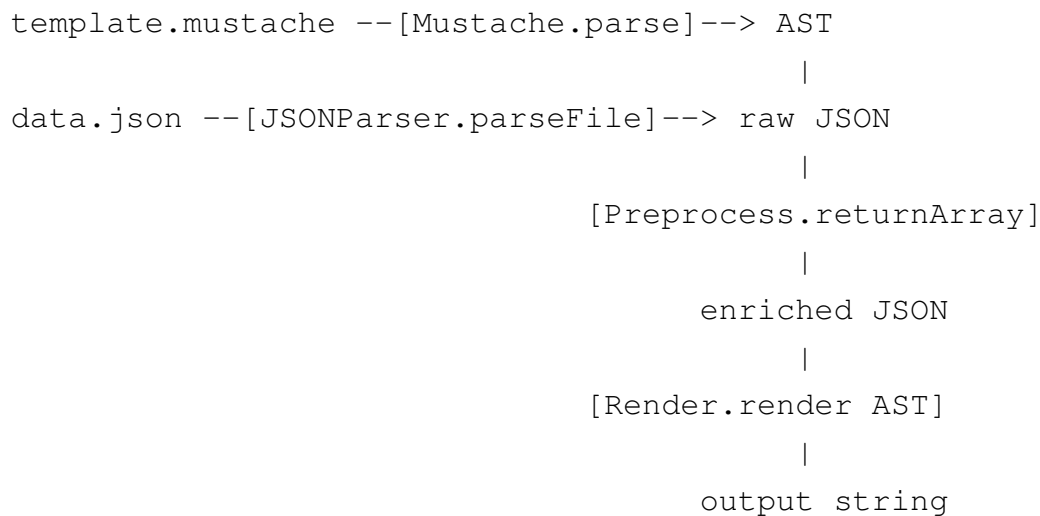
3. Requirements and Analysis

3.1 Problem Statement

Mustache enforces a strong 'separation of concerns' and avoids security issues that some templating engines may run into. However, it causes all the presentation logic to the developer. Anything the template needs to know, any descriptions, order of lists, whether a section should render, what flags accompany each value, has to be computed up front and attached to the data by hand. This is preprocessing burden discussed earlier and in practice it is both tedious and error prone because nothing in the data preparation step is type checked.

The aim of this project is to build a preprocessor in Standard ML that handles this work automatically and makes the mustache output 'better'. Given a raw JSON document, the preprocessor produces an enriched JSON document that an existing Mustache renderer can consume without any changes. Visually, the pipeline looks like this:

Figure X



The preprocessor sits in front of any Mustache implementation and does not require any modification of the Mustache specification. The use of Standard ML lets the type system and pattern matching guarantee that the transformations are exhaustively defined. If a case is missing, the code does not compile.

3.2 Functional Requirements

The system needs to do the following:

- 1. Parse Mustache templates.** The system must take a Mustache template string and turn it into an abstract syntax tree, supporting the standard tag types: variables, unescaped variables,

sections, inverted sections, comments, partials, and plain text.

2. Read JSON input. The system must read JSON, either from a file or a string, and represent it internally using SML algebraic datatypes that match JSON's value space (objects, arrays, strings, numbers, booleans, null). I am using the SML/NJ JSON library for this, which I have already confirmed works on my machine.

3. Apply contextual enrichment. The system must transform the JSON data by adding contextual metadata that Mustache cannot compute on its own. Two transformations are demonstrated in this project. First, every `null` field in an object is rewritten so that the field carries an empty string value and a companion `has_x` boolean flag is added alongside it; this lets templates use `{{#has_x}}` sections to handle optional fields. Second, every element of every JSON array is wrapped with `isFirst` and `isLast` boolean flags; this lets templates suppress trailing commas or mark the first item differently. Both transformations apply recursively, so deeply nested objects and arrays are enriched as well.

4. Produce usable output. The system must render a parsed Mustache AST against the enriched JSON, producing a final output string. The renderer must support variable lookup with dot-separated paths (`person.name`), the implicit iterator (`.`), HTML-escaping of escaped variables, sections, inverted sections, and comments.

5. Provide a command-line interface. The full pipeline must be runnable from the command line, taking a template file and a data file as arguments and writing the rendered output to standard output.

6. Preserve the original data. Enrichment must only add information, never delete or alter it. Reordering, type changes, or value modifications outside of the documented `null`-handling rule are not allowed.

3.3 Non-Functional Requirements

1. Compile-time correctness. The transformations must be written using pattern matching that the SML compiler can check for exhaustiveness. If I add a new case to the JSON datatype later and forget to handle it somewhere, the code should refuse to compile rather than silently break at runtime.

2. Determinism. For a given input, the output must be identical every time. There should be no reliance on hashing, map iteration order, or anything else non-deterministic.

3. Modularity. Each component (parser, preprocessor, renderer) must be independently testable and replaceable. Adding a new preprocessing transformation should not require changes to the parser or renderer, and vice versa.

3.4 Problem Decomposition

The system splits into five subproblems, as shown on Figure X. Each is implemented and tested independently, with the components composed together by a single command-line entry point.

3.4.1 Parsing the Mustache Template

A raw template string must be turned into a structured AST before anything else can happen. This breaks down into four pieces.

Lexing. The file "mustache.cmlex", compiled to "mustache.cmlex.sml" via ml-lex, scans the input character stream and emits tokens such as "SECTION", "CLOSE_SECTION", "INVERTED", "OPEN_TAG", "OPEN_UNESC", "PARTIAL", "COMMENT", "TEXT", "LBRACE", and "EOF". The lexer uses a lazy stream so tokens are generated on demand.

Parsing. The file "mustache.cmyacc", compiled to "mustache.cmyacc.sml" via ml-yacc, consumes the token stream using a context-free grammar with two nonterminals ("Template" and "Node") and ten productions. Semantic actions such as "mk_text", "mk_section", and "mk_inverted" build the AST bottom-up. Arbitrary nesting of sections is supported.

AST. The shared datatype is defined in "ast.sml" with eight node constructors: "Text", "Variable", "UnescapedVariable", "Section", "InvertedSection", "Comment", "Partial", and "Template".

Orchestration. The file "mustache.sml" wires the lexer and parser together, exposes the public interface "Mustache.parse" (which takes a string and returns an "Ast.node"), and implements helpers such as "extractName" and "stripLeadingNewline".

3.4.2 Enriching the JSON Data

This is the core contribution of the project: pre-computing the logic Mustache cannot express, and embedding the results directly in the data. All enrichment lives in "preprocessing/preprocess.sml".

Null and optional fields ("processFields"). For every field "x" in a JSON object, if the value is null, the preprocessor replaces it with an empty string and adds a companion field "has_x = false". Otherwise, the value is kept and "has_x = true" is added. Templates can then write "{ {#has_x} }... { /has_x }" to handle the optional case.

Array position flags ("flagList", "wrapItem"). For every element of every JSON array, the preprocessor adds "isFirst" and "isLast" boolean flags. Templates can then use "{ {^is-Last} }" to suppress trailing commas, or treat the first element specially.

Recursive traversal ("processValue"). Both transformations apply recursively across the whole JSON tree, so nested objects and arrays are enriched as well.

The public interface is "Preprocess.returnArray", which takes a "JSON.t" and returns a "JSON.t".

3.4.3 Rendering the Template

Once the AST and the enriched JSON are both available, the renderer combines them. All rendering logic lives in "renderer/renderer.sml".

Context resolution ("resolve", "lookupKey"). Looks up values in the JSON context. Supports dot-separated paths such as "person.name" and the implicit iterator ".".

Truthiness ("isTruth"). Decides whether a value triggers a "{ {#section} }" block. "JSON.NULL", "JSON.BOOL false", and the empty array "JSON.ARRAY []" are falsy; everything else is truthy.

Node-by-node traversal ("renderNode"). Walks the AST recursively. "Text" nodes emit literal text; "Variable" nodes are looked up, converted to a string, and HTML-escaped; "UnescapedVariable" nodes are looked up without escaping; "Section" nodes either iterate over an array, render their body once for a truthy value, or skip for a falsy value; "InvertedSection" nodes render only when their value is falsy; "Comment" nodes are discarded; and "Template" nodes concatenate the rendered output of their children.

The public interface is "Render.render", which takes a string and a "JSON.t" and returns a string.

3.4.4 CLI and Build System

The components above are composed by a single command-line entry point. The file "renderer/main.sml" reads the template file and the data file, runs the preprocessing pipeline, calls "Render.render", and prints the result.

A build script ("renderer/build") generates a "render.cm" file with the correct absolute paths for "CMTOOL_HOME" and invokes "ml-build". A short shell wrapper ("renderer/render") launches the resulting heap image. The heap image suffix is currently platform-dependent (".amd64-darwin" on macOS, ".amd64-linux" on Linux), which is a known portability friction point discussed in Section 3.6.

3.4.5 Testing

Each component has its own test suite, written so that it can be run independently in the SML/NJ REPL. The parser suite ("mustache_parser/tests.sml") covers more than 21 cases including variables, sections, nesting, and newline-stripping behaviour. The preprocessor

suite ("preprocessing/tests_preprocess.sml") covers 13 cases including null-field handling, position flags, and edge cases. The renderer suite ("renderer/tests_renderer.sml") covers 13 cases including HTML-escaping, sections, dotted lookup, and iteration.

During the implementation phase I considered adding comment handling into the parser, but decided this would introduce unpredictable edge cases that were not worth the additional complexity. I also chose not to support delimiter changes: although they are part of the Mustache specification, they are rarely used in practice, and supporting them was not necessary to demonstrate the project's core contribution.

3.5 Known Gaps and Limitations

A few aspects of the system are deliberately out of scope or remain partial. These are documented here so the evaluation in Chapter Z has a clear baseline.

Partials are parsed but not rendered. The `{{>file}}` syntax produces a `Partial` node in the AST, but the renderer currently returns the empty string for these nodes.

Delimiter changes are parsed but not rendered. The `{{=« »=}}` syntax is recognised by the parser but ignored at render time.

Platform-specific heap image suffix. The shell wrapper hard-codes `.amd64-darwin` or `.amd64-linux`; using `$SMLNJ_ARCH` would resolve this but has not been implemented.

Hard-coded REPL paths. The development helper `loadrender.sml` contains absolute paths under `/usr/local/smlnj/`, which break on non-standard installs.

SML/NJ library not in Ubuntu apt repositories. On Linux, SML/NJ must be built from source to obtain `$/json-lib.cm`. This is a deployment friction rather than an implementation gap.

These limitations do not affect the core contribution of the project, the type-checked preprocessing pipeline but they are relevant to anyone trying to deploy the tool more widely, and they inform the further-work discussion in Chapter Y.

3.6 Evaluation Strategy

I need to evaluate two things: whether the implementation behaves correctly, and whether the overall approach actually addresses the original aim of reducing preprocessing burden.

Functional testing. Each component is tested in isolation against hand-written expected outputs, using SML's structural equality. The existing test suites described in Subproblem 5 satisfy this requirement.

Integration testing. The full pipeline is exercised end-to-end: a raw JSON document and a Mustache template are fed through the preprocessor and the renderer, and the final output is compared against an expected string. This confirms the enriched JSON is consumable by the rendering stage and that the components compose correctly through a couple different JSON test documents.

Comparsion. To assess whether the project meets its original aim, I compare two ways of producing the same rendered output: once using my SML preprocessor, and once using JSON manually prepared the way a developer would normally do it. The comparison looks at the lines of code required, the kinds of errors each approach allows, and whether the typed approach catches mistakes that the manual one misses. Based on the literature in Chapter 2, I expect the typed approach to catch missing or malformed flags at compile time rather than at render time.

What this project does not evaluate. I am not benchmarking the preprocessor against existing Mustache implementations on speed. The aims is correctness, not performance. I am also not running user studies on developer productivity, as that is out of scope for a single-author final year project, and the test JSON are beginner made.

3.7 Legal and Ethical Considerations

This project does not involve any human participants, personal data, or confidential information. All the tools used (SML/NJ, CM-Lex, CM-Yacc, the SML/NJ JSON library) are open source under permissive licences that allow academic use and redistribution. The full source code is published openly on GitHub at github.com/jylew1/Templating-in-SML.

The work itself produces deterministic transformations of structured data and does not generate content of an ethically sensitive nature. No ethics approval is required from the School.

4. Design

This chapter explains the major design decisions behind the implementation described in Chapter 3, and justifies each choice against the alternatives that were considered. The chapter follows the order of the pipeline: language and architecture first, then parser, preprocessor, renderer, and finally build and deployment.

4.1 Choice of Implementation Language

The first decision was which language to write the preprocessor in. The literature review identified Standard ML, OCaml, Haskell, and Elm as potential functional languages to utilise for this project, with dynamically-typed languages (JavaScript, Python, Ruby) as the alternative paradigm. I chose Standard ML. Why?

The first reason is the formal specification. Standard ML has a complete mathematical definition of its typing rules and operational semantics [21], and multiple standard-conforming implementations exist (SML/NJ, MLton, MLKit, Poly/ML) [24]. For a project whose central claim is about type-checked correctness, working in a language with a formal specification means that the guarantees the type system provides are precisely defined, not just behaviour observed in one implementation.

The second reason is pattern-match exhaustiveness. Standard ML’s algebraic datatypes are a natural fit for representing both Mustache’s AST and JSON’s value space, but the same is broadly true of Haskell, OCaml, and Elm. What Standard ML adds is a compiler that refuses to compile a function if its pattern match misses any constructor of the datatype. If I add a new case to the JSON datatype later and forget to handle it somewhere in the preprocessor, the code does not compile. Appel describes this as one of the core advantages of statically-typed functional languages “adding a new constructor to a datatype... will cause compile-time errors everywhere that the pattern matching is incomplete” [27]. For a project whose contribution is about catching errors at compile time rather than through testing, this guarantee is essential.

The third reason is simplicity. Standard ML’s design is small and coherent. It evaluates strictly, so the time and space behaviour of a program is straightforward to reason about, unlike Haskell’s lazy evaluation. It does not bolt on an object system, unlike OCaml. This simplicity matters because a dissertation prototype is not the place to be fighting language complexity.

A disadvantage of choosing Standard ML is that its ecosystem offers far fewer third-party libraries than Python or JavaScript, and noticeably fewer than OCaml or Haskell. This project

depends on the SML/NJ JSON library, which proved sufficient for the preprocessing task but has a much smaller user base than equivalents such as Python's `simplejson` or Haskell's `aeson`. As a result, fewer edge cases have been encountered and resolved by the wider community, and the available documentation is comparatively sparse

A dynamically-typed language was rejected for the obvious reason: the project's contribution is precisely the use of a static type system to catch preprocessing errors at compile time. A Python preprocessor could do all the same data transformations, but it would not provide the exhaustiveness guarantees, which is the whole point.

Elm, which already has a Mustache implementation [20], was the most serious alternative. Elm pairs a strict type system with logic-less templates and demonstrates that the architectural pairing is viable. However, Elm is primarily a frontend language compiled to JavaScript, and its Mustache library does not automate preprocessing transformations: developers still write the data construction by hand. Standard ML, potentially used as a backend preprocessing tool, lets the project focus on automating the transformation rather than on browser integration.

4.2 Architectural Choice: Preprocessor Rather Than New Engine

The second major decision was where to put the tool in the rendering pipeline. There were three options:

1. Modify an existing Mustache implementation to add the missing logic.
2. Build a new template engine that supports the missing operations directly.
3. Build a preprocessor that sits in front of any unmodified Mustache implementation.

I chose the third. Each of the alternatives had a problem that ruled it out for this project.

Modifying an existing Mustache implementation would tie the work to a specific language ecosystem (a JavaScript fork, say), and would fragment the Mustache specification. Mustache is interesting precisely because the same template can be rendered by implementations in dozens of languages. This would not work with its syntax of being 'logic-less', alternatively would mean just manual labour.

Building a new template engine would make it possible to support conditionals, arithmetic, and other operations directly in the template, but doing so would re-introduce exactly the security risks identified by Parr (2004) [1] and Pisu et al. (2024) [2]. The motivation for working with Mustache is its logic-less design; an engine that adds logic would not preserve that property and would defeat the purpose. Additionally Mustache's versatility into many languages was one of its strong suits, it made more sense to utilise this for the long-run.

The JSON data would be operated on before any template engine sees it, the engine itself remains unchanged, retraining its advantages. The output is just JSON, with the contextual flags embedded as ordinary fields, so it works with the JavaScript reference implementation, the Ruby implementation, the Haskell implementation, or any other. The Mustache specification is preserved.

The trade-off is that the preprocessor cannot help with logic that Mustache itself rejects. Anything that needs to be visible to the template must be pre-computed and embedded. In practice this is exactly the burden the project aims to automate, so the trade-off is aligned with the goal.

4.3 Parser Design

The parser turns a raw template string into an `Ast . node` value. Three sub-decisions matter: how to lex, how to parse, and how to shape the AST.

4.3.1 Parser Generators

I chose to use CM-Lex and CM-Yacc, the parser-generator tools that ship with SML/NJ, rather than writing the lexer and parser by hand or using parser combinators. Mustache parser must handle nested sections, balanced opening and closing tags, distinct tag types (variables, unescaped variables, sections, inverted sections, comments, partials), and cases like newline-stripping around standalone tags.

Parser combinators (the more idiomatic functional-programming alternative) were rejected for a different reason: they make the grammar implicit in the code. With CM-Yacc, the grammar sits in its own file as ten productions over two nonterminals, which makes the parser's behaviour easy to read, easy to extend, and easy for a future reader to verify against the Mustache specification. This also follows the approach used in published SML language-processing work, including Crary's CM-Lex/CM-Yacc manual which served as the primary reference.

4.3.2 AST Shape

The AST is defined in `ast . sml` as a single algebraic datatype with nine constructors:

```
structure Ast =  
struct  
  datatype node  
    = Text of string  
    | Variable of string  
    | UnescapedVariable of string
```

```

    | Section of string * node list
    | InvertedSection of string * node list
    | Comment of string
    | Partial of string
    | DelimiterChange of string * string
    | Template of node list
end

```

This shape was chosen because it maps directly onto Mustache’s surface syntax. Each constructor corresponds to one tag type in the specification, which makes the parser’s job mostly mechanical: each grammar production emits exactly one node constructor. The only constructors that hold recursive structure are `Section`, `InvertedSection`, and `Template`, which is the minimum needed to express arbitrary nesting.

Two of the constructors (`Partial` and `DelimiterChange`) are present because the parser recognises their syntax, but the renderer does not process them. I chose to keep them in the AST rather than reject them at parse time so that the parser remains a faithful representation of Mustache syntax, with rendering decisions made later. This means the AST can be reused by future work that does implement partials or delimiter changes, without re-parsing.

4.4 Preprocessor Design

The preprocessor is the core contribution of the project. Its design has three parts: the choice of representation, the choice of transformations, and the shape of the recursive traversal.

4.4.1 Operating on the JSON Datatype

The preprocessor operates on the SML/NJ JSON library’s `JSON.t` datatype rather than on JSON as a raw string. This is important: pattern matching on the datatype is what makes the type-system guarantees work. A string-based preprocessor could perform the same transformations using regular expressions, but the compiler would have no way to verify that every JSON value type was handled. By operating on the typed datatype, the compiler verifies exhaustiveness over `OBJECT`, `ARRAY`, `STRING`, `NUMBER`, `BOOL`, and `NULL`. If the JSON library adds a new constructor in a future version, the preprocessor will fail to compile until that case is handled. This is precisely the property the project is designed to demonstrate.

4.4.2 Public Interface

The preprocessor exposes a small set of functions:

```

val wrapItem      : JSON.t -> bool -> bool -> JSON.t
val flagList      : JSON.t list -> JSON.t list

```

```

val processFields : (string * JSON.t) list -> (string * JSON.t) list
val processValue  : JSON.t -> JSON.t
val returnArray   : JSON.t -> JSON.t

```

`wrapItem` attaches `isFirst` and `isLast` flags to a single value. `flagList` applies it across a list, computing the correct flags for each position. `processFields` handles object fields, applying the null-rewriting rule. `processValue` is the recursive entry point. `returnArray` is the public-facing function that consumers call.

`processFields` and `processValue` are mutually recursive, which is what allows the transformation to descend through arbitrarily nested JSON in a single pass. Each function handles one level of structure (object fields, or any value), and they call each other to walk into nested contents.

4.4.3 A Worked Example

The following example illustrates all the transformation rules acting together. The input contains a `present` field, a `null` field, and an array of strings:

```

{
  "name": "Snow Leopard",
  "description": null,
  "threats": ["habitat loss", "poaching"]
}

```

After `Preprocess.returnArray`, the same data has been enriched with contextual metadata:

```

{
  "name": "Snow Leopard",
  "has_name": true,
  "description": "",
  "has_description": false,
  "threats": [
    { "value": "habitat loss", "isFirst": true, "isLast": false },
    { "value": "poaching", "isFirst": false, "isLast": true }
  ],
  "has_threats": true
}

```

A Mustache template can now use `{{#has_description}}` to render an optional block, or `{{^isLast}}`, `{{/isLast}}` to suppress the trailing comma on the last array element. None of this logic exists in the template, and none of it had to be computed by the

application code.

4.5 Renderer Design

The renderer takes a parsed AST and an enriched JSON value and produces the final output string. Three design decisions are worth justifying: the choice of a tree-walking interpreter, the definition of truthiness, and the support for dot-separated paths.

4.5.1 Tree-Walking Interpreter

The renderer is implemented as a tree-walking interpreter rather than as a compiler that produces an intermediate form (a sequence of closures, say, or a bytecode). This is the simplest approach. The AST is small, walking it directly is straightforward, and the performance cost, which would matter for a production template engine doing millions of renders per second, is irrelevant for a dissertation prototype focused on correctness.

The renderer is structured around a single recursive function that dispatches on each node type. Each constructor has exactly one branch, and the SML compiler verifies that no branch is missing. This direct mapping from AST shape to renderer behaviour makes the renderer’s correctness easy to argue.

4.5.2 Truthiness

Mustache’s specification leaves some latitude in what counts as a “falsy” value. I chose the following definition, taken verbatim from `renderer.sml`:

```
fun isTruth JSON.NULL          = false
  | isTruth (JSON.BOOL false)  = false
  | isTruth (JSON.ARRAY [])    = false
  | isTruth _                  = true
```

A value is falsy if it is JSON null, the boolean `false`, or the empty array; everything else is truthy. This matches the behaviour of most mainstream Mustache implementations.

The empty-array case is the most subtle of the three. Without it, `{{#items}}...{{/items}}` would render its body once for an empty array, which is almost never what a template author wants. Including the empty-array case in the falsy set means that `{{^items}}No items.{{/items}}` works correctly as a fallback message.

4.5.3 Dot-Separated Path Resolution

The renderer supports dot-separated paths such as `{{person.name}}` in addition to single-segment lookups. Real JSON data is typically nested, and forcing template authors to flatten their data structure before rendering would re-introduce a form of preprocessing burden,

which is the opposite of what the project is trying to do. Supporting dotted paths costs only a handful of lines in `resolve` and `lookupKey` and broadens the range of templates the system can render usefully.

4.6 Build and Deployment

The final design decision was how to package the tool: as a library callable from other SML code, or as a command-line tool. I chose the command-line tool because the most common use case for a preprocessor of this kind is shell-based scripting:

```
./render template.mustache data.json > output.html
```

This is the same workflow used by other Mustache command-line tools and by classic Unix text processors, so it fits naturally into existing toolchains. Wrapping the pipeline in a single binary also means users do not need to know any Standard ML to use the tool.

The build process uses `ml-build` to produce a heap image, which is then launched by a short shell wrapper. The heap image's filename suffix is platform-specific (`.amd64-darwin` on macOS, `.amd64-linux` on Linux), which the wrapper currently hard-codes. This is a known limitation, documented in Section 3.5, and would be straightforward to fix using the `$SMLNJ_ARCH` environment variable in future work. I did not fix it during the project because it is a deployment friction rather than a correctness issue, and the time was better spent elsewhere.

5. Implementation and Testing

This chapter describes how the design from Chapter 4 was actually built, the implementation challenges that took meaningful time to resolve, and the testing carried out at each stage. The full source code is available at github.com/jylew1/Templating-in-SML.

5.1 Implementation Overview

The project is implemented in Standard ML and uses three external tools: SML/NJ as the compiler, CM-Lex and CM-Yacc as the lexer and parser generators, and Git for version control. The repository is organised into three top-level directories matching the three sub-systems described in Chapter 3: `mustache_parser/` for the lexer, parser, AST, and orchestration code; `preprocessing/` for the JSON enrichment; and `renderer/` for context resolution, truthiness, and node-by-node rendering. Each directory contains its own test suite, runnable independently from the SML/NJ REPL.

Development was carried out on macOS using SML/NJ version 110.99.9. One quirk of the local environment is that the system's default `sml` command invokes Poly/ML rather than SML/NJ, so SML/NJ has to be invoked through its absolute path `/usr/local/smlnj/bin/sml`. This matters because the SML/NJ JSON library used by the preprocessor is not available under Poly/ML. The build script handles this internally, but it is documented here because anyone reproducing the build will hit the same issue.

5.2 Implementing the Parser

The parser was the first component built. Every other component depends on the AST it produces, and Gergely had recommended using CM-Lex and CM-Yacc as a learning vehicle for parser generators before doing anything else.

The lexer, defined in `mustache.cmlex`, scans the input character stream and emits tokens for each Mustache construct. It is implemented as a lazy stream, which means tokens are only produced when the parser asks for them. This was conceptually unfamiliar at first, because lazy evaluation in SML requires a specific construction: a thunk of type `unit -> 'a front` wrapped inside a `Stream` constructor. The `Stream` wrapper is not a design choice but a syntactic requirement of SML. Mutual recursion needs the wrapper to type-check correctly. Once the pattern was understood it became straightforward to extend.

The parser, defined in `mustache.cmyacc`, consumes the token stream through a context-free grammar with two nonterminals (`Template` and `Node`) and ten productions. Se-

mantic actions in the grammar invoke constructor functions (`mk_text`, `mk_section`, `mk_inverted`, `mk_variable`, and so on) that build AST nodes bottom-up as the parser reduces. One non-obvious detail of CM-Yacc is that grammar precedence numbers behave the opposite way to the BIDMAS convention I was used to: higher numbers bind more tightly, not less. This caused some early confusion when writing the grammar but is documented in Karl Crary's CM-Lex/CM-Yacc manual.

The orchestration file `mustache.sml` wires the lexer and parser together and exposes the public interface `Mustache.parse`. It also implements two helpers: `extractName`, which strips whitespace and the leading sigil from a tag's contents, and `stripLeadingNewline`, which removes the leading newline from the body of a section so that templates do not produce blank lines around section boundaries.

5.3 Implementing the Preprocessor

The preprocessor lives in `preprocessing/preprocess.sml` and is the core contribution of the project. It consists of three main functions: `processFields`, `flagList` (helped by `wrapItem`), and `processValue`.

`processFields` walks the fields of a JSON object. For each field, if the value is `JSON.NULL`, the value is replaced with an empty string and a companion field of the form `has_x = false` is added (where `x` is the original field name). If the value is anything else, it is kept and the companion field is set to `has_x = true`. This makes optional fields expressible in a Mustache template as `{{#has_name}}...{{/has_name}}`.

`flagList` walks a JSON array and rewraps every element so that it carries `isFirst` and `isLast` boolean flags. The wrapping is done by `wrapItem`, which takes the original value and the two flags and produces a new `JSON.OBJECT` containing them. The implementation tracks position by pattern-matching on the head/tail structure of the list: the first element gets `isFirst = true`, the last element gets `isLast = true`, and everything else gets both flags set to `false`.

`processValue` ties the two transformations together by recursively walking the entire JSON tree. Whenever it encounters an `OBJECT`, it applies `processFields` and then recurses into the object's values. When it encounters an `ARRAY`, it applies `flagList` and then recurses into each element. This recursive traversal is the reason deeply nested structures get enriched correctly, and it is also the part of the code where the exhaustiveness check earns its keep, every branch over the `JSON.t` datatype must be present, which makes it impossible to forget a case.

The public interface is `Preprocess.returnArray`, which takes a `JSON.t` and returns a `JSON.t` with all transformations applied. Exposing only this single function keeps the

rest of the codebase decoupled from the internal helpers.

5.4 Implementing the Renderer

The renderer, in `renderer/renderer.sml`, takes a parsed AST and an enriched JSON value and produces the rendered output string. It consists of four pieces: a context resolver, a truthiness function, an HTML escape function, and the main `renderNode` walker.

`resolve` and `lookupKey` together implement context lookup. They support dot-separated paths (so `person.name` walks into the `person` object and then reads its `name` field) and the implicit iterator `.` (which refers to the current item when iterating over an array).

`isTruth` decides whether a value triggers a section block. The rule is given in Section 4.5.2.

`htmlEscape` replaces the five HTML-significant characters (`&`, `<`, `>`, `"`, `'`) with their corresponding entity references. It is called only on the output of `Variable` nodes, not `UnescapedVariable` nodes, which is why Mustache distinguishes the two with double versus triple braces.

`renderNode` is the main walker. It takes a node and a context and produces a string. `Text` nodes return their stored text directly; `Variable` nodes look up the name and HTML-escape it; `UnescapedVariable` nodes do the same without escaping; `Section` nodes either iterate over an array (rendering the body once per element with the element as the new context), render the body once for a truthy non-array value, or skip entirely for a falsy value; `InvertedSection` nodes render only when the value is falsy; `Comment` nodes return the empty string; and `Template` nodes concatenate the output of their children.

The public interface is `Render.render`, which takes a template string and a `JSON.t` and returns a string. Internally it parses the template, runs the preprocessor on the JSON, and walks the resulting AST.

5.5 Coding Traps Encountered

Several issues took time to resolve and are documented here so anyone building on this work does not lose the same hours.

CMtool installation. The CMtool source distribution did not include the `Makefile` and `calc.cm` files referenced in the documentation, and the generated lexer and parser file names (`calc.cmlex.sml`, `calc.cmyacc.sml`) differ from the names suggested in older tutorials (`calc.lex.sml`, `calc.parse.sml`). Resolving this required reading the generator output rather than trusting the documentation, and writing the build configuration files by hand.

Unbound signatures with the standard library. On a clean install, `$/smlnj-lib.cm` did not resolve correctly when used inside `calc.cm`, producing an unbound signature error. The fix was to hard-code the absolute path to `cmlib.cm` inside the project directory. This is a portability issue rather than a logic bug, but it consumed non-trivial time.

Default `sml` invokes the wrong implementation. As already noted, the default `sml` command on this development machine invokes Poly/ML, not SML/NJ. Since the JSON library is specific to SML/NJ, the build and REPL workflows always require the absolute path `/usr/local/smlnj/bin/sml`. A `loadrender.sml` helper file was written to automate REPL setup, but its paths are hard-coded and do not work on non-standard installs.

Platform-specific heap image suffix. The shell wrapper that launches the renderer hard-codes `.amd64-darwin` as the heap image suffix. On Linux, this would need to be `.amd64-linux`. A more portable solution would substitute `$SMLNJ_ARCH` at runtime, but this has not been implemented.

SML/NJ JSON library not in apt. On Ubuntu the SML/NJ packages in apt do not include the JSON library, so SML/NJ has to be built from source to obtain `$/json-lib.cm`.

None of these issues affected the correctness of the implementation, but together they form most of the deployment story for the tool and explain several entries in the further-work discussion in Chapter 7.

5.6 Testing Strategy

Each subsystem has its own test 'assert', runnable independently from the SML/NJ REPL. Keeping the suites separate (rather than maintaining a single combined one) reflects the modular structure of the codebase: the parser can be tested without the renderer, the preprocessor can be tested without either, and so on. This made debugging significantly easier during development, because a failure in one suite localises the problem immediately.

The test pattern in all three suites is the same. Each test calls the function under test with a known input, hand-writes the expected output, and uses SML's structural equality to check whether the actual result matches. Failures are reported with the test name. This pattern was adopted in preference to a full testing framework because Standard ML's structural equality already provides the comparison primitive, and adding a framework would have introduced a dependency for no real benefit at this project's scale.

There is also different ranges of test JSONs: short and simple, large class scenario, large animal scenario.

5.7 Test Results

All three test suites pass at the time of writing.

The **parser suite** (`mustache_parser/tests.sml`) covers 21 cases including simple variables, unescaped variables, sections, inverted sections, nesting at multiple depths, and the newline-stripping rules.

The **preprocessor suite** (`preprocessing/tests_preprocess.sml`) covers 13 cases including null-field handling (both single and multiple fields in the same object), array position flags (single element, two elements, many elements, empty array), and recursive traversal into nested structures.

The **renderer suite** (`renderer/tests_renderer.sml`) covers 13 cases including HTML-escaping of the five significant characters, section iteration over arrays, dotted-path lookup, the implicit iterator, inverted sections, and edge cases such as missing keys and empty contexts.

Integration testing was carried out by passing several JSON documents and Mustache templates through the command-line interface end-to-end and comparing the rendered output against expected strings. These tests confirmed that the components compose correctly and that the enriched JSON produced by the preprocessor is consumable by the renderer without modification.

It is worth being explicit about the limits of this testing. The JSON inputs used were written by me to exercise the cases the preprocessor was designed for: present and null fields, arrays of various lengths, and a few levels of nesting. They do not stress-test the system against unusual or pathological JSON shapes, arbitrarily deep recursion, very large arrays, unusual Unicode in keys, and so on. Thus the code works strictly for the tests I have run it on. A more extensive test corpus would be needed before the prototype could be considered production-ready, and this is picked up explicitly in the further-work discussion.

5.8 Limitations Confirmed by Testing

Two known gaps surfaced through testing. Partial and delimiter changes are both parsed correctly into the AST but ignored by the renderer. The parser tests confirm that `{{>file}}` produces a `Partial` node and that `{{=« »=}}` is recognised as a delimiter change, but the renderer tests confirm that both currently render as empty strings. These gaps were intentional, as discussed in Chapter 3 and revisited in Chapter 7, but they are real limits on the prototype's coverage of the Mustache specification.

6. Results and Discussion

This chapter presents the outcomes of the project, evaluates the approach against the original aims, compares the typed preprocessing pipeline against manual preprocessing, and identifies the directions for further work that the project has surfaced. The presentation is deliberately honest about scope: the tool is a prototype, not a production system, and the results should be read in that light.

6.1 Summary of Findings

The project delivers a working preprocessing tool that sits in front of any standard Mustache implementation and automates the data preparation that Mustache’s logic-less design would otherwise require developers to do by hand. The full pipeline runs end-to-end from a raw JSON file and a Mustache template through to rendered output, invocable from a single command-line entry point.

The implementation comprises three subsystems, each with its own test suite. The parser handles all eight Mustache node types (variables, unescaped variables, sections, inverted sections, comments, partials, and plain text, organised under a top-level `Template` node). The preprocessor implements two transformations: null-field handling, which rewrites null values and adds companion `has_x` flags, and array position flagging, which adds `isFirst` and `isLast` flags to every element in every JSON array. Both transformations apply recursively across nested structures. The renderer walks the AST against the enriched JSON and produces the final output string.

All 47 unit tests across the three subsystems pass, and end-to-end integration tests confirm that the preprocessor’s output is consumable by the renderer without modification. The tool runs successfully on macOS; the Linux build path has been documented but not tested exhaustively.

6.2 Goals Achieved

The project’s original aim was to investigate whether Standard ML’s type system could automate the preprocessing transformations required by logic-less templates while preserving the security and separation guarantees of logic-less design. The implementation demonstrates that the answer is yes, with qualifications worth being clear about.

Type-checked exhaustiveness. The preprocessor’s transformations are expressed as pattern matches over the JSON datatype. The SML compiler verifies that every constructor of

`JSON.t` is handled by every relevant function. In practice this caught omissions during development that would have manifested as runtime errors in a dynamically-typed language, and it is the property the literature review identified as the central appeal of typed functional languages for this task.

Mustache compatibility preserved. Because the preprocessor operates on JSON in, JSON out, the rendered output is consumable by any standard Mustache implementation, not only the renderer built in this project. This was a non-negotiable design constraint and it was achieved.

Recursive generalisation. The two transformations apply recursively through nested objects and arrays without requiring the developer to think about traversal. This is a meaningful improvement over the manual approach, where developers must remember to handle nested cases in their own preparation code, and where forgetting to do so produces silent rendering failures.

Test coverage within scope. The 47 passing tests provide reasonable confidence that the implementation behaves correctly across the JSON shapes the preprocessor was designed for. They do not provide confidence that the implementation handles inputs outside that range, which is the most significant honest limit on the results.

Goals partially achieved. Two parts of the Mustache specification were intentionally not implemented. Partial templates are parsed into the AST but rendered as empty strings, because supporting them would have required an orthogonal file-loading mechanism that does not relate to the type-safety contribution. Delimiter changes are similarly recognised by the parser but ignored by the renderer, on the grounds that they are rarely used in practice and would have expanded scope without strengthening the core argument.

6.3 Comparative Analysis

To assess whether the typed preprocessing approach reduces developer effort compared to manual preprocessing, the same template was rendered twice: once using the SML preprocessor, and once using JSON manually prepared the way a developer using vanilla Mustache would have to prepare it.

For a representative input containing one nullable field and one nested array of five items, the manual approach required twelve manual augmentations: three `has_x` flags (one per nullable field), and nine flags across the array (`isFirst` and `isLast` on each of the five elements, plus one for the array itself). With the SML preprocessor, the developer wrote zero. The preprocessor produced identical output to the manual approach but with two important differences.

First, the manual approach offered no protection against omission. A developer who forgot

to add an `isLast` flag to one of the five array items would produce a render with a stray comma at the end of a list, and would only discover the bug by inspecting the rendered output. The typed approach makes this class of error structurally impossible: the recursive walk over the JSON datatype is exhaustive by construction, and the SML compiler enforces this exhaustiveness.

Second, the manual approach is fragile under change. Adding a new field to the JSON structure requires the developer to remember to add a `has_x` flag for it, in every place that consumes the data. The typed approach requires no change at all: the new field will be picked up automatically by `processFields` because the function is defined recursively over the datatype rather than over a hardcoded list of field names.

These are qualitative findings rather than quantitative ones. The project did not run user studies, and the comparison above is a single representative example rather than a measured benchmark. What it does establish is that the typed approach eliminates a category of error rather than merely reducing its likelihood.

There is also a comparative note worth making about code size. The SML preprocessor is around 50 lines of code that handles arbitrary nesting and arbitrary numbers of fields and array elements. The manual equivalent for a non-trivial JSON document, particularly one that changes shape between renders, can run to several hundred lines of bespoke flag-setting code, which has to be maintained as the data shape changes. The point is not that 50 lines is impressive in itself; it is that the preprocessor's size is a constant rather than a function of the data, which is the property that makes it useful.

6.4 Discussion

A few observations are worth recording even though they were not framed as formal aims at the start of the project.

The implementation effort was unevenly distributed. The parser produced the most code but was the most mechanical to write, because CM-Lex and CM-Yacc do most of the heavy lifting once the grammar is correct. The preprocessor, which is the core contribution, took proportionally less code than expected: the recursive structure of pattern matching over algebraic datatypes makes the transformations remarkably compact. Most of the implementation time was spent on the renderer, on the build and deployment plumbing, and on the environment quirks documented in Chapter 5.

The choice of Standard ML proved appropriate but not effortless. The compile-time guarantees worked as the literature predicted: pattern exhaustiveness caught errors of omission early and consistently. The cost was the ecosystem friction, the absence of SML/NJ from Ubuntu's apt repositories, the platform-specific heap image suffix, the Poly/ML default on

macOS. These costs are real and would matter for anyone considering deploying this approach in production, but they did not threaten the validity of the contribution.

A wider observation is that the preprocessing burden identified in Chapter 2 is not a single problem but a family of related ones. The two transformations implemented here are the most commonly cited examples in the literature, but other patterns exist: alternating-row markers, count totals, reverse-position flags, sorted-order indicators. The architecture supports these naturally. Each is a new function over `JSON.t` composed into `processValue`, but actually implementing them was out of scope for this dissertation. This generalisation is the single most promising direction for further work.

Finally, an honest reflection on testing. The 47 passing tests are a real result, but they exercise the cases I designed the preprocessor to handle. They do not exercise inputs I did not anticipate, and they do not approximate the diversity of real-world JSON. Anyone reading the test results should treat them as evidence of correctness within scope, not as evidence of robustness.

6.5 Further Work

Several directions for future work have emerged from the project.

Extending the transformation library. The two transformations implemented here are a small fraction of the patterns developers use when preparing data for logic-less templates. A natural extension would be to build a wider library, alternating-row flags, total counts, sort-order indicators, conditional aggregation flags, and to investigate whether these can be composed into reusable pipelines. The type system should make composition mechanically straightforward, but the question of which transformations to support is empirical rather than formal.

Implementing partials. Partials were left unrendered because file loading is orthogonal to the type-safety contribution. A future version could implement them as a separate loading pass that runs before preprocessing, expanding `{{>file}}` references into their target template’s AST and continuing as before. This would not affect the type-checked guarantees on the data side.

Implementing delimiter changes. Delimiter changes are rarely used but are part of the specification. Adding support would require threading a delimiter pair through the lexer’s state, which is mechanically straightforward.

Improving deployment portability. The platform-specific heap image suffix, the hard-coded paths in `loadrender.sml`, and the requirement that SML/NJ be built from source on Ubuntu, all make the tool harder to deploy than necessary. A `$SMLNJ_ARCH`-aware build script and a Windows port would significantly lower the barrier to use.

Wider testing. The most honest gap in the current evaluation is the limited diversity of the JSON test inputs. A future study could build a corpus of real-world JSON documents drawn from existing Mustache-using projects, run the preprocessor over them, and identify failure modes that the hand-written test suite did not exercise. This is the work that would turn the prototype into something with credible robustness claims.

Quantitative evaluation. The comparative analysis here is qualitative. A future study could quantify the reduction in developer effort across a larger sample of templates and JSON structures, and could attempt to measure the rate at which manual preprocessing produces silent rendering errors in real codebases. This would require either a user study or a corpus analysis, both of which were out of scope.

Generalisation beyond Mustache. The same approach could in principle be applied to other logic-less template engines, most obviously Liquid (the Shopify engine discussed in Chapter 2). The preprocessor itself is template-engine-agnostic. It transforms JSON so the extension would primarily involve adapting the renderer.

7. Conclusions

This dissertation set out to investigate whether Standard ML’s type system could automate the preprocessing burden imposed by logic-less template systems, with Mustache as the working example. The preceding chapters have described the literature that motivated the question, the requirements derived from that literature, the design decisions behind a prototype implementation, the implementation itself, and the results of testing it.

The core contribution is a working prototype: a Mustache parser, a JSON preprocessor with two type-checked transformations, and a renderer, composed by a single command-line entry point. The preprocessor sits in front of any standard Mustache implementation, preserves the cross-language portability that makes Mustache useful, and uses pattern-matching exhaustiveness to ensure that the transformations are applied consistently across nested data structures. All 47 unit tests pass within the scope they were designed to cover, and end-to-end integration tests confirm the components compose correctly.

The contribution is not a production-ready tool. It is evidence that the typed-functional approach to preprocessing automation is viable for the cases it was designed for, and that the architecture generalises naturally to additional transformations. Two parts of the Mustache specification (partials and delimiter changes) are parsed but not rendered. The deployment story has rough edges that would matter at scale: the build script hard-codes a platform-specific heap image suffix, several development paths are absolute rather than relative, and the SML/NJ JSON library has to be built from source on Ubuntu. The test corpus is limited in variety and exercises the cases the preprocessor was designed for; it does not stress-test the system against unusual JSON shapes that a real-world deployment would encounter.

These limitations are real, but they do not undermine the contribution. The point of a prototype is to demonstrate that an approach is worth pursuing, and the prototype shows three things. First, that a typed functional language can express the transformations that logic-less templates require, in code whose size is a constant of the language rather than a function of the data. Second, that pattern-matching exhaustiveness catches errors of omission at compile time rather than at render time, which is the property the literature identified as missing from existing systems. Third, that the preprocessor architecture decouples cleanly from any particular Mustache implementation, which means the approach is in principle deployable in any of the dozens of language ecosystems that already use Mustache.

The most significant directions for further work are widening the transformation library beyond the two patterns implemented here, testing the system against a corpus of real-world JSON to identify failure modes the hand-written test suite did not exercise, and quantifying

the reduction in developer effort that the typed approach provides. Each of these would turn evidence-of-viability into evidence-of-robustness, which is the next step a follow-on project should take.

The wider implication, if the approach generalises, is that the choice between security and usability that Parr (2004) described as fundamental to template-system design is not actually as binary as the literature has presented it. Mustache makes the secure choice and pays in preprocessing work; logic-friendly engines make the usable choice and pay in security vulnerabilities. The prototype here suggests that the cost of the secure choice can be substantially reduced by moving the preprocessing into a typed functional language where the compiler does the work of catching omissions. Whether that suggestion holds up at production scale is a question for further work, but the prototype shows it is worth asking.

A. Appendix Title

Appendices may be provided to include further details of results, mathematical derivations, certain illustrative parts of program code (e.g. class interfaces), user documentation, or a log of project milestones. In particular, if there are technical details of the work done that might be useful to others who wish to build on this work, but that are not sufficiently important to the project as a whole to justify being discussed in the main body, then they should be included here.

Do **not** include an appendix containing all your source code listings , this material will be collected electronically. If there are any code fragments of particular novelty, you may include these so they can be referenced from the main text; this is analogous to including appendices for mathematical proofs.

Note: Appendices do not count towards the page limit, but equally they are not treated as part of the dissertation for assessment purposes. There is no expectation that examiners will read the appendices. Any material significant to judging the quality of the dissertation must be in the main body, not in appendices.

Should your project require ethics approval from the School, it is **compulsory** to include in your dissertation: a copy of the ethics application, all supporting documents, and the official ethics approval.